


**Proyecto Fénix**

**Daniel Alejandro Díaz Jorigua**

**Juan Esteban Parra Rodríguez**

**Samuel Esteban Linares Hernández**

**Jimena Hernández Rozo**

**Sesión casos de prueba**

**establecido.**

**Centro de Diseño y Metrología**

**Análisis y Desarrollo de Software**

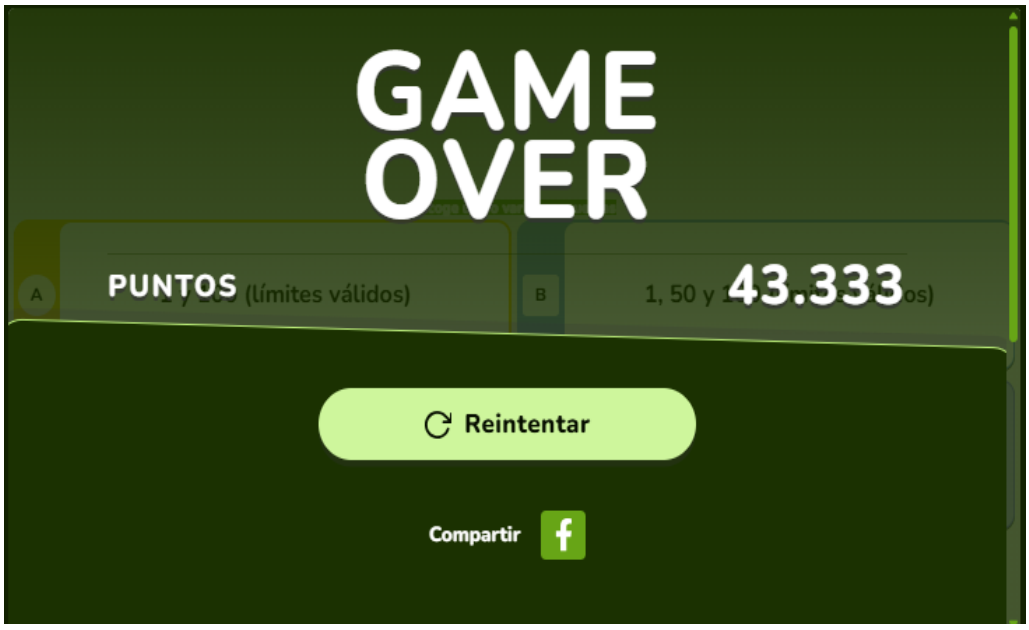
**2026**

**Ejercicio Rápido individual**

**Juan Esteban Parra**




**Jimena Hernández Rozo**



**Pruebas unitarias (según el lenguaje del proyecto)**

| Entorno | Herramienta | Nota |
|---|---|---|
| Java | JUnit | Es la de la clase (assertEquals) |
| Python / Django / Flask | Pytest | Sintaxis simple |
| JavaScript / Node / React | Jest o Vitest | Jest es el más usado |
| API REST (cualquier lenguaje) | Postman o Insomnia | Sirven para probar endpoints como agendar cita, login o listar barberos |

**Pruebas de sistema (caja negra, extremo a extremo)**

Selenium: automatiza el navegador y soporta varios lenguajes.

Cypress: es fácil de empezar y tiene muy buena documentación. Es una buena opción si el proyecto es web.

Playwright: es moderno y rápido, y prueba en Chrome, Firefox y Safari.

Katalon Studio (versión gratuita): permite grabar y reproducir pruebas sin programar mucho, útil si el equipo tiene poca experiencia.

JMeter: sirve para pruebas de carga y rendimiento, que la sesión menciona como parte de la prueba de sistema.

Lighthouse (en Chrome): mide rendimiento, accesibilidad y usabilidad.

**Pruebas unitarias**

django.test.TestCase (viene con Django, basado en unittest): es la base. Cada prueba corre en una transacción que se revierte, así que no ensucia la base de datos. Se ejecuta con python manage.py test.

pytest + pytest-django: es opcional, pero tiene sintaxis más corta y es la herramienta que menciona la sesión para Python.

coverage.py: mide qué porcentaje del código cubren las pruebas. Es útil para justificar el avance en la entrega.

factory_boy (opcional): crea datos de prueba como barberos, clientes y servicios sin escribirlos a mano.